X6 T1 Subset sum problem (SSP) Let us have a set of integers $\{a_1, a_2, \ldots, a_n\}$. In the subset sum problem, the question is whether it is possible to select a subset of these numbers such that the sum of its elements is exactly S, i.e., a certain integer.

What is the time complexity of solving this problem? How do you analyze it?
Does this problem belong to class P or NP?

Counting subsets: each of n elements has two options, it's included to subset or not, set of integers can form 2^n subsets, that gives 2^n possibilities → exponential growth. Brute force checks all 2^n subsets, and no broad pattern lets skip without guessing. There's no shortcuts SSP solving time below exponential in general.

Nondeterministic – polynomial time to check if the sum is S.
Guess 0...n elements → n
Calculate the sum of elements (n-1 operations) → ~n
Test if sum equals S → 1
n + n-1 + 1 = 2n → O(n) epädeterministisesti polynominen aika

Exponential time, $O(k^n)$ where base k>1 is fixed constant
P = **P**olynomial time, n^k where k is fixed constant, e.g. $O(n^2)$ or $O(n^1)$ (also linear is polynomial)
NP = **N**ondeterministic **P**olynomial time, solutions are easy to veryfy in polynomial time if given a candidate answer, but solving may be hard

X6 T2. Based on the analysis of the previous task, calculate the execution times for solving the subset sum problem when the number of integers is 2, 5, 10, 20, 30, 50, 75, 100, 200 and the number of operations per second you can perform is 100, 10,000, or 1,000,000. Report the result in days.

n integers
x = operations per second
SSP has 2^n subsets to check (brute force)

runtime (seconds): 2^n / x
runtime (days): 2^n / x / (24×60×60)

n	2^n	100/s	10,000/s	1,000,000/s
2	4	4.63E-7	4.63E-9	4.63E-11
5	32	3.70E-6	3.70E-8	3.70E-10
10	1,024	1.18E-4	1.18E-6	1.18E-8
20	1,048,576	0.121	0.00121	1.21E-5
30	1,073M	124.2	1.242	0.0124
50	1.126E+15	130M	1.30M	13.0
75	3.778E+22	4.37E+15	4.37E+13	4.37E+11
100	1.268E+30	1.47E+23	1.47E+21	1.47E+19
200	1.607E+60	1.86E+53	1.86E+51	1.86E+49

exceeds 1 day

Comparison: Universe's estimated age ~13.8 billion years, (1.38 * 10^10)*365 = 5.037*10^12 days (5.037E+12)

4.6e-11 = 4.6×10^(-11) = 0.000000000046
1.86e+50 = 1.85×10^50 = 185000000000000000000000000000000000000000000000000

X6 T3. SAT Satisfiability problem. Choose a propositional logic formula with n variables. Denote each of these by x_i, i = 1…n. Such a formula could be, for example $(x_1 \land \neg x_2) \lor \neg(x_1 \land x_3 \lor x_4)$. The satisfiability problem asks whether there exist values $\{x_1, x_2, \ldots, xn\}$, x_i = T|F such that the formula evaluates to true? What is the time complexity of obtaining a solution? How do you analyze it?

Assume that testing one variable assignment in satisfiability problem takes 0.03 seconds.
How many variables can you deterministically check a) in one hour b) in 24 hours c) in one week?

Each variable x_i has options T or F. With n variables, there are 2^n possible combinations. The number of combinations is exponential, so it doesn't belong to class P. To solve whether there's an assignment making the formula true, the simplest algorithm would try every possible truth value combination. Nondeterministically: guess values $\{x_1, x_2, \ldots, x_n\}$, assigning T or F to each, and compute the result. By giving truth values, checking if the formula evaluates to true is quick → correctness of solution can be verified in polynomial time → NP

Time	Seconds	assignments (33.33×sec)	2^n with max n	Max n	2^n+1
1 hour	3,600	120,000	65,536	n=16	2^17 = 131 072
24 hours	86,400	2,880,000	2,097,152	n=21	2^22 = 4 194 304
1 week	604,800	20,160,000	16,777,216	n=24	2^25 = 33 554 432

max n = floor($\log_2$(assignments))
$\log_2(120000) = 16.872...$
floor($\log_2(120000)$) = 16

Assignments per second = 1/0.03 = 33.33 (since each assignment takes 0.03 s)
2^n, total truth assignments needed for the n
max n, largest n that can be fully solved
2^16=65K, 2^21=2M, 2^24=16M assignments
if available assignments < 2^n → cannot guarantee solution

X6 T4. Study the LL(1) parsing presented in the LAP_LL1.pdf file available on elearn. Based on the description, interpret whether all LL1 conditions are met for the following grammars or if some remain unmet.

a)
S → A c | B d
A → a | S e
B → a

(indirect) left recursion
S → Ac → Se

overlapping First-set
FIRST(S) = {a}
FIRST(A) = {a}
FIRST(B) = {a}

S → Ac and A → a
S → Bd and B → a

Since there's no nullable production, there is no FIRST/FOLLOW conflict. FIRST/FOLLOW conflicts can only happen for a nullable production. There is no ε-production in the grammar, so no production has ε in its FIRST set. But follow sets for this grammar are
FOLLOW(S) = {e,$}
FOLLOW(A) = {c}
FOLLOW(B) = {d}

b)
S → a A | b B
A → c D
D → d | ε
B → e

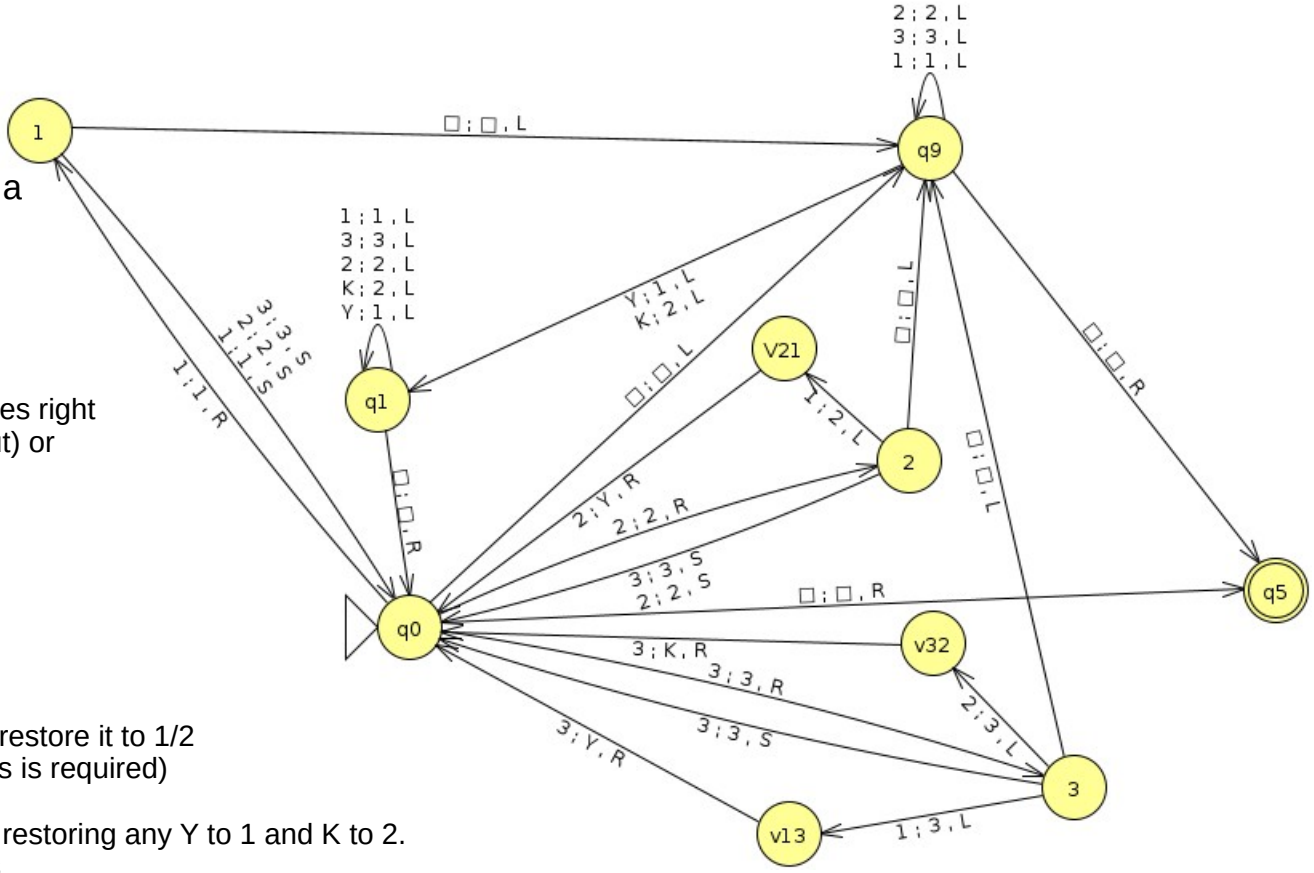
FIRST(S) = {a,b}
FIRST(A) = {c}
FIRST(B) = {e}
FIRST(D) = {d,ε}

FOLLOW(S) = {$} start symbol
FOLLOW(B) = {$} from S→bB, B at end of production, so propagates FOLLOW(S)
FOLLOW(D) = {$} from A→cD, D at end, propagates FOLLOW(A), and FOLLOW(A) gets {$} from S→aA
FOLLOW(A) = {$} from S→aA, A at end

No overlapping First sets
no First/Follow concflict
no left recursion
→ LL1

X6 T5. In the following Turing machine, the input alphabet is {1,2,3} and the tape alphabet is {1,2,3,Y,K}. Can you deduce what the machine does? You can of course build the machine in JFLAP and test it if you want.

Bubble sort process on symbols 1,2,3 – repeatedly scans input, swaps adjacent elements if they are in the wrong order. Markers Y and K are used to remember that a swap happened during a pass so TM can restart another pass. It accepts once full leftward scan finds no markers (tape is sorted).



q0 → 1/2/3 reads current symbol (1,2 or 3) and moves right (on blank, it either accepts immediately (empty input) or goes to q9 to finish a pass

states 1/2/3 handles the case ”read 1/2/3”

v21 swaps 2 and 1
v13 swaps 1 and 3
v32 swaps 3 and 2

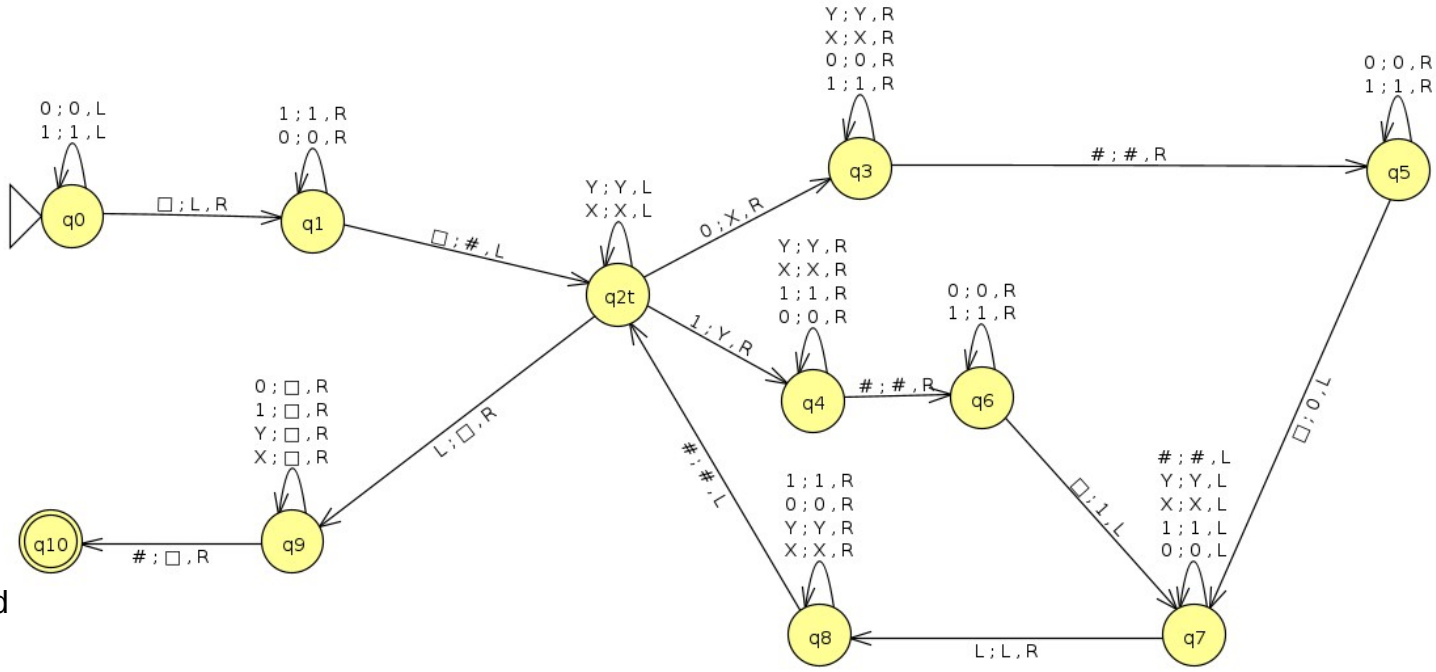
q9 end-of-pass scan back to left. If encounters Y/K, restore it to 1/2 and go q1 (meaning a swap happened, another pass is required)

q1 left-reset state. Moves left to left-end blank while restoring any Y to 1 and K to 2. On blank moves right back to q0 to start a new pass

q5 accepting halt

X6 T6. In the following Turing machine, the input alphabet is {1,0}, and the tape alphabet is {1,0,L,#,X,Y}. What does the automaton do, what is its purpose? The automaton can be found as a file in elearn (H6T6.jff)

TM reverses binary string by repeatedly marking the leftmost unprocessed bit, copying the bit to the far right of a separator, and looping until all bits are copied. After building reversed output it erases the original (marked) input.



q0→ q1 write left-end marker L
q1→ q2t write separator #
q3/q4 after marking 0/1, scan right to separator #
q5/q6 pasting #, go to right end of the output area, write 0/1 and move left
q7 move left accross output area, separator and marked input to left-end marker L
q8 move right, from L to #
q2t→ q9→ q10 cleanup
q10 accept when cleanup reaches #